

Chinese NGCC Algorithms: The First Week of AI Cryptanalysis

Markku-Juhani O. Saarinen
Tampere University, Finland
markku-juhani.saarinen@tuni.fi

Abstract—On September 20, 2026, the Chinese Institute of Commercial Cryptography Standards published the 119 first-round candidates of its Next-generation Commercial Cryptographic Algorithms (NGCC) program: 34 signature schemes, 41 key encapsulation mechanisms, 9 key exchange protocols, and 35 hash functions. The program follows the template of the open NIST competitions for AES, SHA-3, and post-quantum cryptography. It opened just after cryptographers’ “Lee Sedol moment” in the summer of 2026, when AI analysis overtook human analysis of the HAWK signature scheme. We report on the first week of a public evaluation effort in which command-line AI coding agents, directed and reviewed by a cryptographer, built a uniform test harness for all submissions, reproduced the known-answer tests of 545 of 577 parameter sets, and produced security reports. In the first week, 191 active findings dated Sep 21–27 were published for 89 candidates, 78 of them rated Critical: universal forgeries, signing-key recovery, trivial hash collisions, broken implicit rejection, and seeds too short for the claimed security level. Most of the initial AI-assisted findings concerned implementations; by the end of the week, independently reproduced AI-assisted attacks from several model vendors also included new design-level key-recovery techniques. We describe the agentic workflow, the review loop and classification ruleset, and the failure modes we observed. We also benchmarked the candidates and explain why the choice of NGCC hash may largely determine final NGCC public-key performance. We discuss what AI-scale analysis means for designing cryptographic competitions.

Index Terms—cryptanalysis, AI agents, post-quantum cryptography, standardization, side channels, vulnerability discovery

1. Introduction

Open competitions have become the accepted way to select cryptographic algorithms. The template was set by NIST’s Advanced Encryption Standard (AES) process (1997–2000): a public call with explicit submission requirements and evaluation criteria in 1997 [1], 15 first-round candidates from 12 countries, public analysis conferences, and a final selection of Rijndael as AES (FIPS 197) [2] justified in a public report [3]. The SHA-3 competition (2007–2012) repeated the recipe for hash functions [4]; 51

of its 64 submissions entered the first round, and the winner Keccak became SHA-3/SHAKE (FIPS-202) [5], [6].

The Post-Quantum Cryptography (PQC) standardization process launched in 2016 [7] accepted 69 “complete and proper” first-round submissions [8]. After three rounds, it produced ML-KEM, ML-DSA, and SLH-DSA (FIPS 203, 204, 205) [9], [10], [11] in 2024, and a fourth round and an additional signature process followed [12], [13], [14]. Community competitions such as CAESAR [15] have adopted a broadly similar model.

These processes invite the public to break the candidates. First-round PQC cryptanalysis circulated through the public mailing list and was collected in NIST’s official comment archive [16]. Later attacks on long-surviving candidates, such as Rainbow [17] and SIDH/SIKE [18] required entirely new insights and deeper analysis than early findings.

1.1. NGCC – China’s PQC & Hash Competition

China’s Institute of Commercial Cryptography Standards (ICCS) announced the Next-generation Commercial Cryptographic Algorithms (NGCC) program on February 5, 2025 [19]. Public-key and hash calls followed on October 9 [20], [21], and draft block cipher requirements on September 4, 2026 [22]. The program follows the NIST competition model:

- draft requirements and evaluation criteria were published for global comment;
- every submission must include a specification, a reference implementation behind a fixed C API, and known-answer test (KAT) vectors produced by official generators;
- candidates are evaluated for “security, performance and other features” over multiple rounds;
- public discussion takes place on open mailing lists.

Submissions closed on June 30, 2026. ICCS published 119 first-round candidates on September 20 [23], [24] (Table 1), compared with 69 for PQC and 51 for SHA-3.

1.2. Emergence of AI Cryptanalysis

For many years there have been works on *ML cryptanalysis*, where the cryptanalyst trains a neural network

TABLE 1. FIRST-ROUND NGCC CANDIDATES BY SUBMISSION TYPE AND UNDERLYING PROBLEM OR PRIMITIVE.

Family	Sign	KEM	KEX	Hash	Total
Lattice-based	11	25	7	–	43
Code-based	5	15	1	–	21
Multivariate	6	–	–	–	6
Isogeny-based	3	1	1	–	5
Hash-based signatures	4	–	–	–	4
Symmetric VOLE-in-the-head	4	–	–	–	4
Other group action	1	–	–	–	1
Hash functions	–	–	–	35	35
Total	34	41	9	35	119

Sign: signatures; KEM: key encapsulation; KEX: key exchange. Code- and multivariate-based MPC/VOLE constructions are counted with their underlying problem.

or another machine learning model directly on plaintext, ciphertext, or key samples. The model is then given a key-recovery or distinguishing task. Examples are Gohr’s neural distinguishers for round-reduced Speck [25] and the SALSA transformer attacks on LWE [26].

A much more recent field is *AI cryptanalysis*, which in its current casual use means using large language models (LLMs) and related methods to *analyze cryptographic algorithms*: to discover and implement cryptanalytic attacks, methods, and the related abstract theory [27].

In other words, ML cryptanalysis attacks a distribution of samples; AI cryptanalysis attacks a specification, a proof, or a piece of code. This paper is about AI cryptanalysis in the second sense, with implementations included in scope.

Pinpointing the “Lee Sedol moment”. In March 2016, AlphaGo [28] defeated top Go professional Lee Sedol, and professionals had to accept that a machine can now play their game better than they did. We observe that many cryptographers had a comparable moment in the July-August of 2026. It centered on HAWK [29], a lattice-isomorphism-based signature scheme that NIST had advanced to the third round of its additional signature process in May 2026 [14].

On June 25, a human-written paper claimed polynomial-time key recovery based on four unverified heuristics [30]. Within days, a SageMath implementation written largely by frontier models showed that success probability collapses with dimension, tracing the failure to one heuristic [31]. On July 28, Anthropic researchers published a Claude-assisted reduction of HAWK- n key recovery to SVP in dimension $n/2 + 1$ [32], [33]. The HAWK designers withdrew the next day [34]. A co-author of the heuristic attack wrote on NIST’s forum: “*This is a historic moment in cryptology: AI has beaten humans at cryptanalysis*”, reporting the lessons that “*humans can make mistakes*” and “*humans are slower than AI*” [35]. Other recent developments include:

- CryptanalysisBench found that five frontier models break 65–86% of a set of known-broken primitives, and uncovered a new key-recovery attack on SpoC and an error in a published security proof [36];

- Anthropic reported an AI-found attack on round-reduced AES [33];
- an LLM-discovered shortcut broke several McEliece key-recovery challenges [37], and further AI-assisted work solved TII-254 [38];

The work reported here continues this line. Once frontier models can analyze a single scheme, the natural next step is to point them at an entire competition. A decade ago, the Brutus study [39] covered all 57 first-round CAESAR candidates and over 210 implementations. Manual specification analysis first found flaws in three candidates, two of which were later withdrawn. Automated testing produced security-usage notes on four submissions and statistical notes on four more. Brutus exercised ciphers through their APIs; it could not read specifications, compare them with code, or assess parameters. Current coding agents can do all three.

1.3. Our Contributions

The NGCC first round is the first large, open cryptographic competition to be evaluated with AI agents from day one. We report on a public evaluation effort that began the evening the candidates were published and made its first reports public 12 hours later (Fig. 1). Our contributions are:

- 1) an *agentic evaluation workflow* for a whole competition: a uniform, source-only build and KAT harness; agent briefs; a vulnerability report schema with severity and evidence levels; and an independent review loop (Section 2);
- 2) a *snapshot of the results* after one week: 191 active findings in 89 of 119 candidates, grouped into recurring defect classes, with the ruleset that classifies them and cycle counts for all candidates (Sections 3 and 5, Appendix A);
- 3) *lessons learned* about where agents help, where they fail, and what this means for competition design (Section 6).

The evaluation project home page is: <https://ngcc.dev>

2. Agentic Evaluation Workflow

One cryptographer directed command-line coding agents working on a single shared repository; OpenAI’s Codex (~\$200 monthly subscription to GPT-5.6 and GPT-6 models) and Anthropic’s Claude Code (~\$100 monthly subscription to Opus 5, Opus 5.5, and Fable 5.1 models). From Sep 21 to 27 (UTC), NGCC work used about 2.01 billion tokens: 1.14 billion in Claude Code (97% cache reads, 5 million output) and 0.87 billion in Codex.

The agents ran semi-isolated on a dedicated cloud server (28 cores, 180 GB RAM) under their own Linux accounts without `sudo`, with internet access, compilers, SageMath, and Lean. Control was via `ssh`; the operator moved data via `scp` and GitHub, and the agents had no git write credentials.

The agents extracted information from specifications, wrote code, reports, and reproduction scripts; the human chose targets, set rules, and made the final decision on every published finding. The workflow has five parts.

2.1. Ingestion and a source-only harness

The 119 submission archives were downloaded, hashed, and unpacked; together they contain 87,514 files. Treating these as untrusted input was the first design decision. A static survey found nothing overtly malicious, but found 471 prebuilt executables and 1,525 prebuilt object files or libraries. The agents’ hard rule: *nothing shipped in a submission is ever executed or linked*, including Makefiles, CMake files, scripts, and binaries. The harness instead compiles each candidate’s reference implementation from its C sources into one shared library per parameter set, using common flags (`-O2 -fwrapv`). The official DRNG and auxiliary functions are linked in, and a linker version script exports only the official API. A KAT driver loads each library into its own process with time and memory limits, replays the official known answer test generators (KAT_KEM, KAT_SIG, KAT_KEX, and KAT_CryptHash) byte for byte, and compares the output with digests of the submitted vectors.

The per-candidate Makefiles were written by agents in batches. Each agent received a brief with four worked examples, the hard rules above, and a time box of about 20 minutes per candidate. The brief also told agents to fix build problems on the harness side, never to relax the harness itself, and never to edit the submission tree; patched copies live in a separate directory.

On day one the harness built 577 instances of 117 candidates, and 545 of them (94.5%) reproduced the submitted KATs exactly. The 32 exceptions were already informative: signing keys that ignore the official randomness source (Galas), vectors produced by a different code revision (Phoenix-SM3, QUBE), a signer that writes past its own claimed signature length (MORNING-ATLAS), and a verifier that accepts a modified message (SQLsign2D2).

2.2. Specification digests

A second brief asked agents to write, for every candidate, compact pseudocode and a parameter table taken from the specification PDF. The agents had to cite algorithm numbers, check specified object sizes against those the compiled library reports, and spot-check constants. The brief states that “pseudocode comes from the specification, not from your memory of similar schemes” and “a short, accurate file beats a long speculative one.” Later design review thus worked from a verified summary, not from 100 pages of PDF.

2.3. Security review

Security analysis proceeded in several passes. Each pass produced candidate-local notes and reproducers:

- *low-hanging fruit*: ciphertext and signature mutation tests, key-equality tests (catching unseeded or fixed-seed generators), and API robustness tests;

- *parameter audits*: seed and prehash lengths compared with the claimed security level, internal entropy audit, and lattice parameters screened with a public lattice estimator [40];
- *specification-versus-code review*: each implementation checked against its specification digest;
- *constant-time review*: 119 scoped notes on secret-dependent branches, table lookups, and divisions in reference code.

Every runtime check in the attack tool is paired with a *control*, a sound candidate for which the check must report NOT-CONFIRMED. The reproduction script fails if any witness disappears or any control fires.

2.4. Vulnerability reporting schema

Each finding has a stable identifier (e.g., `sign-32-1` for the first finding of signature scheme #32) and eight required fields: severity, status, scope, affected instances, discovery effort, exploitation, credit, and date.

Severity runs from Info to Critical. Critical means a demonstrated violation of a primary security claim or NGCC target, such as forgery, key recovery, or an IND-CCA2 break. By cryptographic convention, practicality is not required: a 2^{256} attack on a 512-bit claim is a “break”. The full ruleset is in Appendix A.

Status separates the strength of evidence:

- *Confirmed*: a runtime or compiled-code witness exists, or the security issue is clear under inspection;
- *Probable*: the defect is established but no attack path has been derived;
- *Lead*: the argument is heuristic and not instantiated;
- *Proof gap*: a security proof does not support its claim.

Scope is one of design, implementation, or side-channel. A schema checker rejects malformed reports, and each finding carries a “Reproducing” section.

2.5. Independent review and publication

A separate reviewer re-ran claims in six rounds, listing required fixes, commands re-executed, and unverified claims. Anthropic models generally reviewed OpenAI output or vice versa. Models can fail to correct their own reasoning errors without external feedback [41]. Cross-vendor review was a workflow choice; confidence came from independently checking evidence, not model agreement alone.

The worker replied in follow-up notes until the items were closed. Reports that cleared review were synchronized to a static web site (<https://ngcc.dev>) and a public harness repository with all reproducers (<https://github.com/ngcc-dev/ngcc-harness>). A forum watcher and a “harvest” procedure imported third-party results from the official NGCC mailing lists, GitHub issues, and ePrint [42], each re-verified locally before publication and credited to its discoverer.

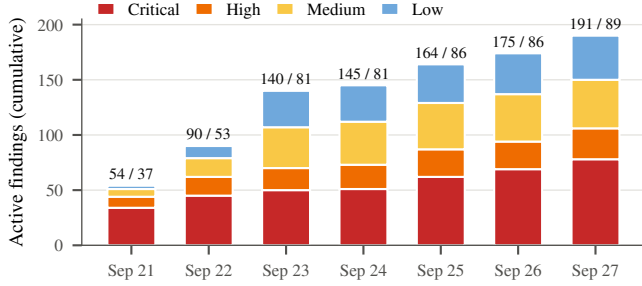


Figure 1. Cumulative active findings by the date shown on each finding, with severities under the final ruleset (bar labels: findings / reports; one Info finding is not drawn). The candidates were published on Sep 20.

TABLE 2. ACTIVE FINDINGS DATED SEP 21–27 BY SEVERITY, DISCOVERY CREDIT, SCOPE, AND CATEGORY. AI INDICATES DISCLOSED ASSISTANCE.

Severity	Scope			Category				Total
	Des.	Impl.	S.-c.	Sign	KEM	KEX	Hash	
Critical	32	46	0	35	24	5	14	78
High	14	13	1	11	12	2	3	28
Medium	24	9	11	14	18	4	8	44
Low	6	17	17	9	13	2	16	40
Info	0	1	0	0	0	0	1	1
Total	76	86	29	69	67	13	42	191
This project + AI	24	59	27	30	45	12	23	110
Others + AI	10	9	1	15	5	0	0	20
Human / unknown	42	18	1	24	17	1	19	61
Candidates with a report				27	32	7	23	89
Candidates in round 1				34	41	9	35	119

3. The First Week

The first reports went public on Sep 21, about 12 hours after work began; 54 findings in 37 reports carry that date. The Sep 28 snapshot holds 195 finding records dated Sep 21–27 in 89 reports; seven dated Sep 27 appeared on Sep 28. Four are withdrawn records retained for their identifiers, leaving 191 active findings (Table 2): 167 Confirmed, 10 Probable, 10 Leads, and 4 Proof gaps. Of the 78 Critical findings, 76 are Confirmed and two Probable; 55 candidates have at least one Critical finding.

Who found what. 110 active findings are credited to the maintainer “with AI assistance”, including 47 of the 78 Critical findings. By scope they are 59 implementation, 27 side-channel, and 24 design findings. The other 81 findings are credited to outside researchers and groups who reported on the NGCC forums, GitHub, the NIST PQC forum, or in a concurrent ePrint [42]. Of those 81, 52 are design findings and 20 disclose AI assistance: ten name OpenAI models (Codex, GPT Luna), two Chinese models (Doubao, Xuanzhi CryptoLLM), and eight none. The other 61 are human / unknown. The second half of the week blurred the earlier division between agent breadth and human design cryptanalysis.

Recurring defect classes. The Critical findings fall into a few families:

- *Verification that does not verify.* UVW’s signature API computes the verification result and then returns 0, so every signature is accepted (sign-32-1). A SQIsign2D2 verifier’s verdict depends on stale stack contents once assertions are compiled out (sign-25-1).
- *Broken Fujisaki–Okamoto transforms and encodings.* In Aegis-Enc+ the implicit-rejection path is dead code. CheetahKEM and LoongKEM use a rejection mask that leaks seven bits of every shared-secret byte (kem-09-1, kem-18-1). Amoeba’s re-encryption check compares only every fourth ciphertext byte, enabling full key recovery (kem-02-1). In five more KEMs, ignored padding or noncanonical encodings let a different ciphertext decapsulate to the same key (kem-06-4, kem-07-2, kem-10-3, kem-31-2, kem-38-4).
- *Missing randomness.* Zero-seeded private generators or skipped DRNG calls give publicly reproducible signing keys (Galas, VDOO) or KEM keys and encapsulations (HEP-QC).
- *Mask, index, and transcript bugs.* A decoder in MORNING-ATLAS overwrites a temporary, so adjacent mask coefficients are equal and two signatures expose the secret key (sign-15-4). In CEDRUS+C, a macro precedence error leaves only four reachable few-time-signature addresses (sign-03-1). Chinith omits the only public-key-dependent value from its final Fiat–Shamir challenge; the ordinary signer run on a false witness forges for any public key (sign-05-1).
- *Seeds and prehashes below the claim.* Many 384- and 512-bit parameter sets expand the entire key from a 256-bit seed, or sign a 512-bit unsalted message digest. Either caps the security at 2^{256} (e.g., kem-11-1, sign-02-1). CreTAKE passes a length in bits where bytes are expected, and absorbs 8 seed bytes instead of 64 in 23 source files (kex-03-1).
- *Hash padding.* Mixed bit orders in Eijen’s padding give $H(\epsilon) = H(0^7)$ (hash-09-1). MasterCube’s pad10*1 collapses at rate boundaries (hash-17-1).

Design breaks. Practical design attacks were mostly credited to outside researchers. One Tins signature yields the signing witness by $\mathbb{F}_2$ elimination on at most 1,044 unknowns (sign-29-1). A Facto-DSA-128 public key yields a universal trapdoor, allowing forgery in 6.4 s (sign-10-2). Tsinghua Hash Lab is credited with full-round MoFang collisions (hash-19-1) and one Neulaser state merger (hash-21-2); Yuan et al. analyze MoFang and a separate Neulaser mechanism (hash-21-1) [42]. A public variable permutation permits public-key-only Origami-128 forgery (sign-18-5).

From day five, Martin Feussner reported signature attacks with OpenAI Codex assistance. Independent witnesses confirmed key recovery and forgeries for Shuttle (sign-23-1), Sigurd from 4-8 signatures (sign-24-1), and ATLAS-128 (sign-15-7), plus public-key-only Lynxer and one-query SQIsignTriangle forgeries (sign-14-1, sign-27-5). The BiT-128 and VDOO-128 attacks remain Probable because their end-to-end searches were not independently reproduced (sign-02-2, sign-33-6); attack provenance and evidence status must be recorded separately. Human researchers added polynomial-time Polar-KEM and DARTS-128 key recovery (kem-29-2, sign-08-2).

Side channels. The week produced 29 side-channel findings: 27 from our constant-time pass (10 Medium and 17 Low under the final ruleset), a Medium rejection-sampling leak in PolarLAC (kem-30-1), and one High retry-reaction finding for UVW-KEM (kem-38-3). An independent review found that the original pass had excluded divisions by compile-time constants, the KyberSlash pattern [43], and compiled every relevant site. Five candidates emit a secret-dependent divide or branch at their default or vendor optimization levels, and many more sites do so at -Os.

4. On Quantum Security Evaluation

Based on our AI-assisted review, none of the 119 candidates rests on a problem with a known polynomial-time quantum algorithm: none relies on integer factoring, discrete logarithms, or other abelian hidden-subgroup problems, and no hybrid includes a classical public-key component. The one commutative group-action design, the isogeny NIKKE NIIKE, faces Kuperberg’s subexponential quantum attack, which its parameters account for. Several candidates do fall to classical polynomial-time attacks (Sec. 3), which trivially carry over to the quantum-circuit model.

The call defines n -bit “quantum-resistant security strength” as known quantum attack complexity of at least 2^n , but leaves the cost metric unspecified. It also states: “The submitted algorithms shall be capable of supporting three classical security strength levels: 128, 256, and 512 bits. The corresponding quantum-resistant security strength shall be no less than 80, 128, and 256 bits.” [44, Sec. 2(1)]. The ICCS feedback justifies 80 bits as sufficient for “interoperability with related symmetric primitives” [45, Sec. 4]. This contradicts a simple (logical) quantum gate count, so it may be intended account for overheads, Grover’s poor parallelism, or circuit depth (cf. NIST’s “MAXDEPTH” [7]). According to design specifications, submitters were divided in two main camps in their reading of the requirement, affecting parameters, performance, and bandwidth:

Under the literal reading, 80 bits means an attack cost of at least 2^{80} , so Grover search requires a 160-bit space at the nominally 128-bit level. Sixteen candidates raise or add parameter sets in response, including the CEDRUS variants, FlexTree, Galas, TRIKE, HARE, and the three SQIsign variants; GreatWall adds a 192-bit set.

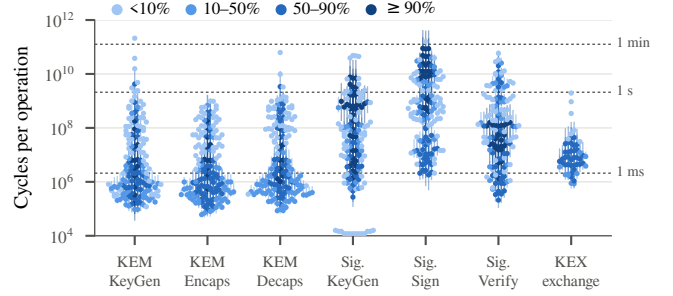


Figure 2. Cycles per operation of KAT-reproducing reference instances (x86-64 at 2.1 GHz; dashed: wall-clock). Width shows how many instances have a given cost; shading is the placeholder share s ; bars illustrate $C(1 - s + sk)$ if helper time scaled like a hash costing $k = 0.17\text{--}4.8 \times \text{pseudoXOF}$ (10-90th percentile of candidates, 1 KiB).

Under the symmetric-equivalence reading, 80 bits is the quantum resistance attributed to a 128-bit symmetric primitive such as SM4 or AES-128, e.g., 2^{64} Grover iterations of a 2^{16} -gate oracle. Five candidates use this explicitly (Weaver, Loom, Qing Luan, Chinith, and UVW-KEM), and ten more implicitly retain 128-bit keys, messages, or seeds.

The camps overlap: Weaver and Loom size hashes literally but justify 128-bit session keys by equivalence. The distinction matters little for the 43 lattice candidates’ attack estimates, but directly for generic search and symmetric components. Ultimately ICCS will decide if the submitters violated the requirement, and how.

5. Placeholder XOFs and Performance

China’s SM3 hash standard [46] has a fixed 256-bit digest and is not an XOF. The call requires placeholder functions (pseudohash and pseudoXOF), to be replaced by new hash/XOF implementations in subsequent rounds [44, Sec. 3.4(5)]. A large symmetric share of the running time is common for NIST PQC algorithms [47], so a benchmark of the submitted code may largely measure the placeholder.

Following the ICCS self-assessment guide, we timed KAT-checked reference builds on one i7-12700 core at 2.1 GHz: 391 instances of 78 public-key candidates (353 of 73 KAT-reproducing, used below) and 140 of the 35 hashes. Link-time wrappers log every ICCS helper call with its lengths. There was also a source survey for symmetric components.

Performance diverges by orders of magnitude (Fig. 2). Even among 128-bit parameter sets, signing ranges from 1.8 million cycles (COMPASS-SIG-128) to 40 billion (TRINE-128-Balanced), and verification spans five orders of magnitude. Hashing 1 KiB takes $0.14\times$ to $111\times$ as long as the placeholder pseudoXOF.

The placeholder often dominates. The median public-key operation spends 36% of its cycles in the ICCS helpers, 403 of 959 operations at least half, and three of the four hash-based signatures 92-99% (13 candidates partly bypass the helpers with their own SHAKE, AES, or SM3).

For many candidates, the hash/XOF selected in the same competition may determine public-key performance almost entirely; the recorded call profiles will let us estimate it per selection.

Possible bandwidth improvements. At the 256-bit level (NIST Level 5), 13 of 38 KEMs have a smaller public key plus ciphertext than ML-KEM-1024, but by at most 29%. Ten of 33 signature schemes undercut ML-DSA-87: the three SQIsign variants are 13–17× smaller at 0.4–2.5 s per signature, and YuanYang-1024 (NTRU hash-and-sign) is 11% smaller than Falcon-1024. Every other smaller signature has a Critical finding, five of them design-level.

6. Lessons Learned

What the agents did well. Agents did the tedious, broad work that a human cannot do for 119 candidates in a week: build files for heterogeneous code, exhaustive sweeps (all 7,168 bit flips, not one sample), runnable witnesses for suspected defects, and line-by-line specification-versus-code checks. 110 of the 191 active findings are marked “trivial” to discover once someone looks, and that is exactly the layer where agent throughput pays off. In this sense, the workflow is Brutus [39] for the 2020s: an automated sanity check across a whole competition, now able to read specifications and run code.

Failure modes. Most of the effort went into review, and the review record lists the failures that matter:

- *Severity inflation.* The first batch had no Medium or Low findings. One report badged as High was described in its own text as a lead. The first fixes were the status field and a rule of “no Critical without a candidate-specific recovery construction”. These grew into a central ruleset tied to the NGCC call’s targets, now part of the agents’ instructions; all findings were re-rated under it (Appendix A).
- *Pattern-matched claims.* The agents called 512-bit parameter sets “NIST Level 5” 19 times, and stated a Gram matrix product in the wrong order. Withdrawn drafts claimed a 2^{256} generic bound where a Joux multicollision gives about 2^{136} .
- *Testing the placeholder.* The NGCC API ships a correctness-only `pseudohash`. Five draft reports attacked this stand-in rather than the scheme and were withdrawn before publication; two published records were withdrawn for the same reason. The ruleset now requires assuming ideal replacements.
- *Hollow coverage.* 95 of 119 constant-time notes disclaimed part of their coverage, and 55 public-key notes had not examined lookups or divisions. A “no issue” note therefore often meant “not examined”. The KyberSlash-type constant divisor was excluded without a stated reason.
- *Non-reproducible reproducers.* Hard-coded paths and a wrong interpreter invocation survived three review rounds before every command actually ran on the host.

The review record shows mostly errors of *scope and calibration* (severity, affected instances, unexamined code), not invented vulnerabilities. In the constant-time pass, for example, the reviewer confirmed that every cited code site existed and that no public value had been reported as a leak. Controls, runnable witnesses, and an adversarial second reviewer were the most effective countermeasures.

What is still human. Through day four, most novel design-level cryptanalysis came from human researchers, some AI-assisted, while the agents provided breadth and re-verification. The later Codex-assisted signature breaks show that this split is already unstable: the reporter attributed the attack work to an AI agent, while human judgment framed the targets and independent review established which claims reproduce. What remains distinctly human in this workflow is responsibility for scope, evidence, disclosure, and the final decision to publish. Whether agents can make those cryptanalytic and epistemic judgments reliably remains open [33], [36].

Implications for competitions. Every NGCC submitter could have run a comparable agentic audit before the deadline, and many of the Critical implementation bugs would likely have been caught. We suggest that future calls provide an official source-only build and KAT harness with guard bytes and negative tests, require submitters to run it before submission, and expect the first round to reach this density of findings within days, not months. Conversely, a flood of fast findings needs a status taxonomy that separates confirmed breaks from leads. It also needs a clear statement that the absence of a report is not a security certificate.

Disclosure. The candidates are first-round submissions, not approved or standardized algorithms, and ICCS published them explicitly for public analysis. All witnesses run against locally compiled submission code; no live system was attacked. As on NIST’s `pqc-forum`, findings were published openly, many via the official NGCC mailing lists where submitters can respond. Several have, and at least one has fixed its reference code.

7. Conclusions

The NGCC first round received in one week a broad, first-pass scrutiny of similar scope that previous (“pre-AI”) competitions accumulated over months. Implementation defects and poor parameter selection still dominate the 191 findings, but the week also produced AI-assisted key recovery against complete designs, with models from several vendors, including Chinese ones. The comparison is no longer AI breadth against human depth, but a pipeline in which discovery, implementation, and verification each mix humans and models.

This also clearly demonstrates not only the offensive but also the defensive side of AI agents; they can’t find bugs that aren’t there. Most algorithm designers and implementers will no doubt soon be familiar with AI agent use, and

software patching is already at an all-time high. Security improves once finding bugs is cheap and fixing them routine.

For NGCC, the most valuable outcome of the first week may be education rather than the number of breaks; second-round teams will know how to avoid such issues. The project will continue tracking NGCC issues independently as attacks become “deeper” and AI models evolve.

LLM Usage Statement

LLMs are integral to the methodology of this work. As described in Section 2, command-line AI coding agents wrote the build harness, the specification digests, security notes, finding reports, and reproduction scripts under human direction. Every published finding was reviewed, and whenever possible re-executed, before publication. All findings carry an evidence status and a reproduction procedure, and the reproducers are public. Using proprietary models limits reproducibility of the *process*, though not of the *results*: each claim is backed by a deterministic witness that runs without any model. LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality. This included drafting text from the project’s records, compiling statistics, and generating figures.

References

- [1] NIST, “Announcing request for candidate algorithm nominations for the Advanced Encryption Standard,” Federal Register, Vol. 62, No. 177, pp. 48051–48058, September 1997. [Online]. Available: <https://www.govinfo.gov/content/pkg/FR-1997-09-12/pdf/97-24214.pdf>
- [2] —, “Advanced encryption standard (AES),” NIST, Federal Information Processing Standards Publication FIPS 197, November 2001. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- [3] J. Nechvatal, E. Barker, L. Bassham, W. Burr, M. Dworkin, J. Foti, and E. Roback, “Report on the development of the Advanced Encryption Standard (AES),” *Journal of Research of the National Institute of Standards and Technology*, vol. 106, no. 3, pp. 511–577, May–June 2001.
- [4] NIST, “Announcing request for candidate algorithm nominations for a new cryptographic hash algorithm (SHA-3) family,” Federal Register, Vol. 72, No. 212, pp. 62212–62220, November 2007. [Online]. Available: <https://www.govinfo.gov/content/pkg/FR-2007-11-02/pdf/E7-21581.pdf>
- [5] S. Jen Chang, R. Perlner, W. E. Burr, M. S. Turan, J. M. Kelsey, S. Paul, and L. E. Bassham, “Third-round report of the SHA-3 cryptographic hash algorithm competition,” National Institute of Standards and Technology, Interagency or Internal Report NIST IR 7896, November 2012.
- [6] NIST, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Federal Information Processing Standards Publication FIPS 202, August 2015.
- [7] —, “Submission requirements and evaluation criteria for the post-quantum cryptography standardization process,” NIST Call for PQC Standardization Candidates, December 2016. [Online]. Available: <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>
- [8] G. Alagic, J. Alperin-Sheriff, D. Apon, D. Cooper, Q. Dang, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, and D. Smith-Tone, “Status report on the first round of the NIST post-quantum cryptography standardization process,” National Institute of Standards and Technology, Interagency or Internal Report NIST IR 8240, January 2019.
- [9] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” Federal Information Processing Standards Publication FIPS 203, August 2024.
- [10] —, “Module-Lattice-Based Digital Signature Standard,” Federal Information Processing Standards Publication FIPS 204, August 2024.
- [11] —, “Stateless Hash-Based Digital Signature Standard,” Federal Information Processing Standards Publication FIPS 205, August 2024.
- [12] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, and D. Smith-Tone, “Status report on the third round of the NIST post-quantum cryptography standardization process,” National Institute of Standards and Technology, Interagency or Internal Report NIST IR 8413-upd1, July 2022, includes updates as of September 26, 2022.
- [13] G. Alagic, M. Bros, P. Ciadoux, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, H. Silberg, D. Smith-Tone, and N. Waller, “Status report on the fourth round of the NIST post-quantum cryptography standardization process,” National Institute of Standards and Technology, NIST Internal Report 8545, March 2025.
- [14] G. Alagic, M. Bros, P. Ciadoux, Q. Dang, T. H. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, H. Silberg, D. Smith-Tone, and N. Waller, “Status report on the second round of the additional digital signature schemes for the NIST post-quantum cryptography standardization process,” National Institute of Standards and Technology, NIST Internal Report 8610, May 2026.
- [15] D. J. Bernstein, “CAESAR: Competition for authenticated encryption: Security, applicability, and robustness,” Web site, 2014. [Online]. Available: <https://competitions.cr.yt.to/caesar.html>
- [16] NIST, “Post-quantum cryptography: Round 1 submissions,” Computer Security Resource Center, 2017. [Online]. Available: <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/Round-1-Submissions>
- [17] W. Beullens, “Breaking rainbow takes a weekend on a laptop,” in *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part II*, ser. Lecture Notes in Computer Science, Y. Dodis and T. Shrimpton, Eds., vol. 13508. Springer, 2022, pp. 464–479. [Online]. Available: <https://eprint.iacr.org/2022/214>
- [18] W. Castryck and T. Decru, “An efficient key recovery attack on SIDH,” in *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, ser. Lecture Notes in Computer Science, C. Hazay and M. Stam, Eds., vol. 14008. Springer, 2023, pp. 423–447. [Online]. Available: https://doi.org/10.1007/978-3-031-30589-4_15
- [19] ICCS, “Announcement on launching the next-generation commercial cryptographic algorithms program (NGCC),” Institute of Commercial Cryptography Standards, February 2025. [Online]. Available: https://www.niccs.org.cn/niccs/Notice/pc/content/content_1937428197396713472.html
- [20] —, “Call for proposals for the next-generation public-key cryptographic algorithms,” Institute of Commercial Cryptography Standards, October 2025. [Online]. Available: https://www.niccs.org.cn/niccs/Notice/pc/content/content_1975896137741635584.html

- [21] —, “Call for proposals for the next-generation cryptographic hash algorithms,” Institute of Commercial Cryptography Standards, October 2025. [Online]. Available: https://www.niccs.org.cn/niccs/Notice/pc/content/content_1975892908773478400.html
- [22] —, “Call for comments on submission requirements for block cipher algorithms (draft) and evaluation criteria for block cipher algorithms (draft),” Institute of Commercial Cryptography Standards, September 2026. [Online]. Available: https://www.niccs.org.cn/niccs/Notice/pc/content/content_2095703042235641856.html
- [23] —, “NGCC: First-round candidate public-key cryptographic algorithms,” Institute of Commercial Cryptography Standards, September 2026. [Online]. Available: https://www.niccs.org.cn/niccs/Notice/pc/content/content_2100875480644800512.html
- [24] —, “NGCC: First-round candidate cryptographic hash algorithms,” Institute of Commercial Cryptography Standards, September 2026. [Online]. Available: https://www.niccs.org.cn/niccs/Notice/pc/content/content_2100880167381061632.html
- [25] A. Gohr, “Improving attacks on round-reduced speck32/64 using deep learning,” in *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II*, ser. Lecture Notes in Computer Science, A. Boldyreva and D. Micciancio, Eds., vol. 11693. Springer, 2019, pp. 150–179. [Online]. Available: <https://eprint.iacr.org/2019/037>
- [26] E. Wenger, M. Chen, F. Charton, and K. E. Lauter, “SALSA: Attacking lattice cryptography with transformers,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 34 981–34 994. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/e28b3369186459f57c94a9ec9137fac9-Abstract-Conference.html
- [27] M.-J. O. Saarinen, “On AI & cryptanalysis: HAWK “guessing game” is not polynomial-time / other recent AI adventures,” Talk, Workshop on Attacks in Cryptography 8 (WAC 8), UCSB, August 2026. [Online]. Available: <https://mjos.fi/doc/20260816-wac8-ai-cryptanalysis.pdf>
- [28] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, January 2016.
- [29] L. Ducas, E. W. Postlethwaite, L. N. Pulles, and W. P. J. van Woerden, “Hawk: Module LIP makes lattice signatures fast, compact and simple,” in *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part IV*, ser. Lecture Notes in Computer Science, S. Agrawal and D. Lin, Eds., vol. 13794. Springer, 2022, pp. 65–94. [Online]. Available: <https://eprint.iacr.org/2022/1155>
- [30] B. Nelson, J. Limbrey, C. Ling, and A. Mendelsohn, “Cryptanalysis of HAWK: a guessing game,” Cryptology ePrint Archive, Paper 2026/1318, June 2026. [Online]. Available: <https://eprint.iacr.org/2026/1318>
- [31] M.-J. O. Saarinen, “HAWK “guessing game” is not polynomial-time,” Cryptology ePrint Archive, Paper 2026/1377, July 2026. [Online]. Available: <https://eprint.iacr.org/2026/1377>
- [32] Z. Straznickas and S. A. Weis, “HAWK- n key recovery reduces to SVP in dimension $n/2 + 1$,” Cryptology ePrint Archive, Paper 2026/1593, August 2026. [Online]. Available: <https://eprint.iacr.org/2026/1593>
- [33] Anthropic, “Discovering cryptographic weaknesses with claude,” Anthropic Blog Posting / “Frontier Red Team”, July 2026. [Online]. Available: <https://www.anthropic.com/research/discovering-cryptographic-weaknesses>
- [34] L. Ducas, “Re: HAWK- n key recovery reduces to SVP in dimension $n/2 + 1$,” pqc-forum mailing list, July 2026, July 29, 2026; withdrawal on behalf of the HAWK team. [Online]. Available: https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/2r2u6SbHun4/m/0_J2KOZ_CQAJ
- [35] C. Ling, “Re: HAWK- n key recovery reduces to SVP in dimension $n/2 + 1$,” pqc-forum mailing list, July 2026, July 30, 2026. [Online]. Available: <https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/2r2u6SbHun4/m/JXBDCsVLBAAJ>
- [36] L. Fluri, A. Shafran, N. Carlini, M. Jagielski, M. Nasr, O. Dunkelmann, E. Ronen, and F. Tramèr, “Cryptanalysisbench: Can LLMs do cryptanalysis?” *CoRR*, vol. abs/2607.18538, July 2026.
- [37] M.-J. O. Saarinen, “HOVER: Higher-order vanishing endomorphism recovery,” Cryptology ePrint Archive, Paper 2026/1778, August 2026. [Online]. Available: <https://eprint.iacr.org/2026/1778>
- [38] —, “Two-anchor holdout/hermite: Solving the TII-254 McEliece key recovery challenge,” Cryptology ePrint Archive, Paper 2026/1986, September 2026. [Online]. Available: <https://eprint.iacr.org/2026/1986>
- [39] —, “The BRUTUS automatic cryptanalytic framework: Testing CAESAR authenticated encryption candidates for weaknesses,” *Journal of Cryptographic Engineering*, vol. 6, no. 1, pp. 75–82, 2016.
- [40] M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of learning with errors,” *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, October 2015.
- [41] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=IkMD3fKBPQ>
- [42] Y. Yuan, R. Wu, S. Wei, J. Shen, J. Liu, and Y. Zhang, “Structural analysis of seven hash functions submitted to the NGCC,” Cryptology ePrint Archive, Paper 2026/2152, September 2026. [Online]. Available: <https://eprint.iacr.org/2026/2152>
- [43] D. J. Bernstein, K. Bhargavan, S. Bhasin, A. Chattopadhyay, T. K. Chia, M. J. Kannwischer, F. Kiefer, T. B. Paiva, P. Ravi, and G. Tamvada, “KyberSlash: Exploiting secret-dependent division timings in Kyber implementations,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 2, pp. 209–234, 2025.
- [44] ICCS, “Submission requirements for public-key cryptographic algorithms,” Institute of Commercial Cryptography Standards, October 2025. [Online]. Available: <https://www.niccs.org.cn/niccs/Notice/IDop1mav.pdf>
- [45] —, “Feedback on the comments received on the draft submission requirements and evaluation criteria for public-key cryptographic algorithms,” Institute of Commercial Cryptography Standards, October 2025. [Online]. Available: <https://www.niccs.org.cn/niccs/Notice/Xi2xTprH.pdf>
- [46] Standardization Administration of China, “Information security techniques—SM3 cryptographic hash algorithm,” General Administration of Quality Supervision, Inspection and Quarantine and Standardization Administration of China, Chinese National Standard GB/T 32905-2016, August 2016, in Chinese. [Online]. Available: <https://openstd.samr.gov.cn/bzgk/std/newGbInfo?hcno=45B1A67F20F3BF339211C391E9278F5E>
- [47] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4,” Second NIST PQC Standardization Conference, Santa Barbara, CA, USA, August 2019, cryptology ePrint Archive, Paper 2019/844. [Online]. Available: <https://eprint.iacr.org/2019/844>

Appendix A.

Vulnerability Report Classification Ruleset

The following is the project’s classification ruleset as of Sep 28, 2026, rendered verbatim from its original Markdown. It evolved during the first week, and the agents follow it when they assign severity, status, and layer (called *scope* in the main text); all published findings were re-rated under it.

REPORT_CLASSIFICATION.md

Vulnerability report classification

This document defines how NGCC findings receive *Severity*, *Status*, and *Layer* labels. These labels answer different questions and must be assigned independently:

- **Severity** describes the demonstrated or bounded security impact.
- **Status** describes the strength of the evidence for the stated finding.
- **Layer** identifies where the vulnerable behavior enters.
- **Remediation burden** describes the cost of removing the behavior. It is normally separate context. For side-channel findings, however, the absence of a portable and reasonably efficient constant-time realization can be evidence that the problem belongs to the design and performance profile.

A finding can therefore be Critical but easy to fix, or Low but expensive to fix. An expensive constant-time implementation is not automatically a design flaw, but it becomes design evidence when the construction’s claimed software performance relies on the leaky operation and no portable competitive constant-time path is supplied or known.

Here *supplied* means the submitters provide the constant-time code, and *known* means a well-known security-engineering technique exists that the report itself can describe (for example masked arithmetic, bitslicing, in-register shuffles, AES instructions, Beneš networks, or a published constant-time decoder or solver). The same meaning applies to hash functions.

Security targets

A finding is judged against the NGCC call as well as the specification’s own claims. The canonical documents are published by ICCS.

Public-key algorithms (call):

- Classical security of 128, 256 and 512 bits, with quantum security of at least 80, 128 and 256 bits, respectively; an optional 384-bit classical level has a 192-bit quantum target (Submission Requirements §2(1)). “Complexity” covers both time and memory (ICCS feedback §5).
- Each signature key pair must support at least 2^{64} different messages of up to 2^{63} bits (§2(2)).
- A KEM’s encapsulated key and a key exchange protocol’s shared secret key must be at least as long as the corresponding classical security level (Submission Requirements §2(2)). An output shorter than its level is a direct NGCC-target shortfall, even when no separate IND-CCA or key-recovery attack is demonstrated.
- Security is evaluated against an attacker with signatures on up to 2^{80} chosen messages or decapsulations of up to 2^{80} chosen ciphertexts (Evaluation Criteria §1(2)). This is an evaluation ceiling for the attacker; the operational per-key requirement remains 2^{64} messages. Classify query-budget findings as follows:
 - **Critical:** a concrete attack costs less than the required level for an attacker using at most 2^{80} queries and within the per-key usage model that the candidate claims or supports. A collision probability or loose proof term alone is insufficient: the report must connect it to a primary break, such as forgery, key recovery, or distinguishing.
 - **High:** despite a broader security claim, the specification’s argument or proof bound covers fewer queries (typically 2^{64}), or becomes too loose at 2^{80} , but no concrete attack below the required level within 2^{80} queries has been shown. Use *Status: Proof gap* unless a specific attack path is identified. Missing or insufficient analysis alone is never Critical.
 - No finding solely from the query limit: the argument covers 2^{80} queries; the query-dependent terms remain negligible at 2^{80} ; or the candidate explicitly limits each key to at least the required 2^{64} uses, makes no broader per-key claim, and no attack is shown within that limit. Record the operational limitation where it matters to interpreting the results.
- Signatures must be EUF-CMA or SUF-CMA; KEMs must be IND-CCA2 (§1(1)).

Hash algorithms (call):

- Collision security of at least $h/2$ bits and preimage and second-preimage security of at least h bits for an h -bit digest, with quantum attacks no cheaper than generic ones (Evaluation Criteria §1(2)).

- Digest lengths of 512 and 1024 bits (768 optional) and message lengths of at least $2^{64} - 1$ bits (Submission Requirements §2(1)). The targets apply to all supported lengths, whatever the specification claims.

Severity

Rate the strongest consequence supported by the report, including its stated preconditions and controls. Do not rate the feared consequence of an uncompleted attack.

Critical

The report demonstrates a violation of a primary cryptographic security claim or applicable NGCC target, or gives a concrete attack bound below that target. Examples include:

- fresh-message forgery, full signing-key recovery, or universal verification;
- challenge-key recovery or a successful IND-CCA, IND-CPA, or key-exchange real-or-random distinguishing attack;
- collisions, preimages, or second preimages below the applicable bound;
- an enumerated key, randomness, or transcript space below the applicable level;
- a KEM or key-exchange output shorter than the classical level required by the call;
- a complete mathematical reduction of the claimed problem to work below the claim, even if that work is not currently practical.

NGCC requires a 512-bit classical level and permits an optional 384-bit level. A demonstrated 2^{256} attack against a 512-bit claim is therefore Critical: practical feasibility is not required when the advertised security bound is violated.

High

The report establishes a serious security failure but stops short of a complete primary-claim break. Typical cases are:

- a candidate-specific recovery/forgery method with one clearly identified engineering or scale step still unvalidated;
- a strong proof or parameter failure with a well-supported path to violating the claim, but without a complete end-to-end witness;
- attacker-controlled memory corruption with credible impact beyond process termination;
- a security argument that covers fewer signing or decapsulation queries than the call's 2^{80} evaluation budget, without a concrete shortfall (see Security targets);
- a measured, repeatable secret-dependent oracle together with a candidate-specific recovery method, where only the observation channel or full-scale run remains open.

A demonstrated violation of a claimed property that the NGCC call does not require is High rather than Critical. For example, signature malleability that violates a claimed SUF-CMA property while leaving EUF-CMA intact is High, because the call accepts either EUF-CMA or SUF-CMA. Failing the designed security level of a required property is Critical.

Do not use High merely because a defect is expensive to repair. A side-channel finding can nevertheless be High when it exposes secrets directly and the construction lacks a portable constant-time realization without a substantial and justified loss of its advertised performance.

Medium

The report confirms a security-relevant defect or secret-dependent leakage, but does not demonstrate recovery, forgery, or a primary indistinguishability break. Examples include:

- a stable decryption-failure or reaction oracle without a completed recovery;
- repeated secret-dependent control flow or memory access for which a plausible local observation model exists;
- a substantive proof, binding, or conformance gap whose concrete attack consequence is unresolved;
- an attacker-supplied public key that forces or repeats an honest party's derived key, without breaking the scheme's honest-key security game;
- a side-channel repair that requires a material algorithmic rewrite or moderate performance/storage cost.

Low

The demonstrated impact is narrow and presently separated from a cryptographic claim violation. Examples include:

- denial of service, resource exhaustion, or an API contract error without memory disclosure or control-flow impact;
- source-level secret dependence with no measured channel or extraction, when a local mask, fixed-bound loop, small-table replacement, or existing constant-time path removes it;
- a limited robustness or interoperability defect;
- a disclosed internal status whose existence alone does not defeat the claimed transform.

An easy fix may support Low when the demonstrated consequence is already narrow. It must not downgrade a demonstrated key recovery, forgery, or claim violation.

Info

Use Info for withdrawn records or non-security tracking entries. A live security finding should normally be Low or above.

Side-channel severity

For timing, cache, branch, division, power, or address-trace findings, answer these questions in order:

1. Is the varying operand secret or derived from secret data at that point?
2. What can observe it: source inspection only, co-resident cache/process, local timing/power, or remote timing/API status?
3. Is the channel measured and repeatable, or only visible in source or machine code?
4. Is there a candidate-specific map from observations to a key, shared secret, plaintext, or forgery?
5. Has that map been executed end to end with the submitted implementation?

Use the resulting consequence, not the mere presence of a branch or table, as the primary severity determinant:

Evidence and consequence	Usual severity
Source-level secret branch/address, no measured channel, local repair	Low
Repeated secret leakage, measured channel, or material constant-time rewrite; no recovery	Medium
Direct fine-grained leakage intrinsic to the advertised fast path, with no competitive constant-time realization supplied or known	High may be justified
Stable oracle plus candidate-specific recovery path, one channel/scale gap remains	High
End-to-end key/secret recovery or forgery	Critical

The table is a default, not a substitute for candidate-specific cryptanalysis. Public-value branches and lookups are not vulnerabilities.

Hash-function portability rule

For this audit, hash inputs and internal states are treated as potentially secret. Hash functions are routinely used on passwords, key material, transcripts containing secret contributions, and other confidential data.

A hash side channel is `Layer: Design` when all of the following hold:

1. the fast construction or the submission's advertised performance relies on secret-indexed tables, secret-dependent control flow, or another operation that is not portable constant-time;
2. removing that operation requires a different evaluation strategy rather than a local mask or canonical lookup replacement;
3. the portable constant-time strategy has a significant demonstrated or technically justified performance cost; and
4. no competitive portable constant-time path is supplied by the submitters or known as a well-known technique.

Hardware instructions may provide a safe fast path on some targets without curing the design-portability problem. A primitive that is constant-time only where a particular instruction-set extension is available is materially different from an ARX-style construction whose ordinary scalar operations are naturally constant-time across common processors.

Under this rule, a large byte-indexed table that is central to a hash's speed can support `High / Design`, even before a complete message-recovery experiment, when the address trace directly reveals message or state bytes and the only portable repairs impose a substantial performance loss. The report must identify the portable repair and measure its cost where feasible; an unmeasured slowdown must be labelled as an estimate.

Status

Status records confidence in exactly the claim made by the report:

- **Confirmed:** directly established by source/specification reasoning, a mathematical certificate, or a reproducible experiment with appropriate controls. `Confirmed` does not imply end-to-end exploitation; the `Exploitation` line must state the demonstrated boundary.
- **Probable:** the vulnerable path and consequence are strongly supported, but an important source, platform, or experimental step remains unchecked.
- **Lead:** a plausible candidate-specific attack or cost reduction still depends on a material unvalidated assumption or experiment.
- **Proof gap:** a stated theorem or reduction is invalid, incomplete, or covers an insufficient query range, but no attack of the claimed strength follows yet.
- **Withdrawn:** the original finding is false, duplicate, out of scope, or superseded. Keep its stable ID and use `Severity: Info`.

Confirmed may describe a confirmed source-level leak rated Low, while a High or Critical attack can remain a Lead. Severity and status must not be collapsed into one axis.

Layer

Assign the layer from the root cause, not from repair difficulty:

- **Design:** the normative construction, parameters, proof, or advertised portable performance profile contains the weakness. The finding applies to conforming implementations, or the submission's fast implementation relies on a construction choice whose portable constant-time replacement has a substantial justified cost. For hashes, apply the portability rule above.
- **Implementation:** the submitted source deviates from a sounder specification or introduces an ordinary API, parser, memory-safety, arithmetic, or randomness defect.
- **Side-channel:** the finding depends on observing timing, branches, cache addresses, division latency, power, faults, or another non-functional execution trace and does not meet the Design criteria above. Use this even when the specification claims the supplied code is constant-time; state that contradiction in the report.
- **Evaluation:** the issue concerns KATs, archive provenance, build metadata, reproducibility, or another evaluation artifact rather than the algorithmic security boundary.

The absence of a constant-time implementation ordinarily proves only that the submitted code is not hardened. To label the root cause *Design*, show that the specification mandates the observable behavior, that the advertised fast path structurally relies on it under the hash portability rule, or that the claimed performance/security profile cannot be retained by a portable constant-time implementation.

Remediation burden

Record remediation in the finding body when it helps evaluation:

- **Local:** a mask, fixed bound, canonicity check, small constant-time lookup, or already-supplied build path; limited expected cost.
- **Moderate:** a published constant-time decoder/solver, an expanded-key or caching change, or another substantial rewrite with credible moderate cost.
- **Architectural:** a change to the key/ciphertext format, core algorithm, or performance profile, or a repair with a measured prohibitive cost.

Use measurements where possible. Qualify an unmeasured cost as an estimate. State tradeoffs: storing an expanded secret changes memory and key handling; masked scans, sorting/permutation networks, dense multiplication, and constant-time elimination may change asymptotic or concrete cost. Do not call such changes local without bounding those costs.

For hash functions, remediation burden also informs the Design classification: a local masked operation or small in-register substitution remains a side-channel implementation defect, while replacing a performance-defining large table or data-dependent evaluation strategy can be a design-level portability defect.

Decision procedure

For each finding:

1. Trace public and secret inputs through the vulnerable operation.
2. State the exact observed behavior and attacker model.
3. Identify the strongest demonstrated consequence and its controls.
4. Compare that consequence with the applicable NGCC target and any explicit specification claim.
5. Assign *Status* from evidence, *Severity* from consequence, and *Layer* from root cause independently.
6. Describe remediation and its expected cost separately.
7. Keep limitations in the report: what was not measured, recovered, forged, or proved.

For contest pseudohash and pseudoXOF, assume ideal replacement primitives with the same external dimensions and parameters. A finding that disappears solely on that replacement is not publishable.